Ejercicio de Aplicación 🤓

L6: Métricas de desempeño de un algoritmo de resresión

Es hora de que pongas en práctica todo lo aprendido. 🤓

Este apartado tiene el objetivo de ayudarte a seguir potenciando tus habilidades, por lo que a continuación encontrarás diferentes **desafíos** que podrás resolver de forma independiente y a tu ritmo.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 😉

¡Manos a la obra!

1. **Desafío** 🎯

   Calcular e interpretar métricas de desempeño para modelos de regresión y clasificación utilizando la librería Scikit-Learn.

   Consigna

➔ *Utilizando los siguientes datos ficticios, calcula las métricas de desempeño para un modelo de* ***regresión****:*

Precio real (USD)	***Precio predicho (USD)***
20000	19500
25000	24000
30000	31000
22000	23000
27000	26500

- *Calcula:* ***MAE, MSE, RMSE y R²***

➔ *Para un modelo de* ***clasificación****, utilizando los siguientes datos:*

Real	***Predicho***
1	***1***
0	***0***
1	***0***
0	***0***
1	***1***

- *Calcula:*
    - ***Matriz de Confusión***
    - ***Precisión***
    - ***Exactitud***
    - ***Sensibilidad***
    - ***Especificidad***

➔ *Calcula y grafica la* ***Curva ROC-AUC*** *utilizando las siguientes probabilidades predichas:*

Real	***Probabilidad clase positiva***
1	***0.90***
0	***0.40***
1	***0.80***

0	**0.35**

1	**0.70**

2. **¿Dónde se lleva a cabo?** 👩‍💻

   Visual Studio Code

3. **Tiempo de dedicación** ⏳

   2 Horas.

4. **Recursos**🛠️

   ➔ *https://scikit-learn.org/stable/modules/model_evaluation.html*

   ➔ https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc

5. **Plus** ➕

   ✅ *Aplicar validación cruzada para obtener un promedio de las métricas.*

   ✅ *Implementar un informe detallado utilizando classification_report.*

6. ⚠️**Condición**

   Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Ejercicio de Aplicación 🤓

L6: Métricas de desempeño de un algoritmo de resresión

Resolución del desafío 👇

```
plt.legend(loc='lower right')import numpy as np
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    confusion_matrix, accuracy_score, precision_score, recall_score,
    roc_curve, roc_auc_score
)
import matplotlib.pyplot as plt

# Datos regresión
y_real_reg = [20000, 25000, 30000, 22000, 27000]
y_pred_reg = [19500, 24000, 31000, 23000, 26500]

# Métricas de regresión
mae = mean_absolute_error(y_real_reg, y_pred_reg)
mse = mean_squared_error(y_real_reg, y_pred_reg)
rmse = np.sqrt(mse)
r2 = r2_score(y_real_reg, y_pred_reg)

print("Métricas de Regresión:")
print(f"MAE: {mae}")
print(f"MSE: {mse}")
print(f"RMSE: {rmse}")
print(f"R²: {r2}")

# Datos clasificación
y_real_clf = [1, 0, 1, 0, 1]
y_pred_clf = [1, 0, 0, 0, 1]

# Métricas clasificación
cm = confusion_matrix(y_real_clf, y_pred_clf)
accuracy = accuracy_score(y_real_clf, y_pred_clf)
precision = precision_score(y_real_clf, y_pred_clf)
recall = recall_score(y_real_clf, y_pred_clf)
specificity = cm[0, 0] / (cm[0, 0] + cm[0, 1])
```

```
print("\nMétricas de Clasificación:")
print(f"Matriz de Confusión:\n{cm}")
print(f"Exactitud: {accuracy}")
print(f"Precisión: {precision}")
print(f"Sensibilidad: {recall}")
print(f"Especificidad: {specificity}")

# Curva ROC-AUC
y_proba_clf = [0.9, 0.4, 0.8, 0.35, 0.7]
fpr, tpr, thresholds = roc_curve(y_real_clf, y_proba_clf)
auc = roc_auc_score(y_real_clf, y_proba_clf)

plt.plot(fpr, tpr, label=f'AUC = {auc:.2f}')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curva ROC')

plt.grid(True)
plt.show()
```